DAG-EVM and Conflict-Graph Consensus for the Lux Multi-Chain

Zach Kelling

Lux Industries

April 2026

Abstract

We present a unified DAG consensus model for the Lux multi-chain that lifts Block-STM-style optimistic concurrency from a per-block scheduler into a first-class finality primitive. Each block-producing chain (C-Chain EVM, D-Chain DEX, Z-Chain ZK-UTXO, A-Chain AI, O-Chain Oracle, R-Chain Relay, S-Chain ServiceNode) emits transaction batches as vertices in a native directed acyclic graph, with conflict sets derived from the chain's natural state model: EVM read/write sets, UTXO nullifiers, DEX order tuples, job identifiers, feed rounds, destination-chain nonces, or service epochs. Antichains are finalised in parallel under the Quasar post-quantum certificate scheme; sequential worldview is preserved by a serialisability theorem. We formalise the safety and no-double-spend properties in Lean 4 and TLA$^+$, prove commutativity of non-conflicting vertices, and show that GPU-native conflict-graph reduction plus GPU ecrecover/keccak yields a 32× practical speedup on the dominant critical-path operations. The result is a DAG-EVM that preserves bytecode-level compatibility with Ethereum, a DAG-Z-Chain that preserves ZK-UTXO double-spend security, and a framework that cleanly contrasts Lux against contemporary parallel and DAG blockchains (Aptos, Sui, Solana, Monad, Fuel).

Contents

1 Introduction

Parallel blockchain execution splits into two orthogonal problem families: *intra-block* parallelism (execute the transactions of a linear block in parallel) and *inter-block* parallelism (finalise non-conflicting blocks in parallel, a partial-order consensus). The industry has focused almost exclusively on the first: Aptos' Block-STM [1], Monad's pipelined execution [3], Sei's parallel EVM, and Polygon Miden's optimistic execution all run in linear-chain consensus. DAG consensus, when present (Sui's Mysticeti [2], Aleph Zero's AlephBFT, Avalanche's Snowman^{++}), typically does not thread the DAG into the state-machine layer: blocks are ordered by the DAG and then executed sequentially, so the DAG exists above the execution engine rather than inside it.

Lux takes the third path: *the execution engine's conflict graph **is** the consensus DAG.* Each VM emits vertices whose parents are determined by the state-level dependency relation of that VM, and the DAG engine (nebula) finalises antichains that by construction contain no mutually-conflicting vertices. The downstream property is **serialisability by construction**: any topological order of the accepted DAG yields identical state, and no two finalised siblings can double-spend, re-write the same EVM slot, or re-submit the same nullifier.

This paper presents:

- A uniform definition of the per-chain conflict rule (§2);
- The DAG-EVM construction and its Block-STM lineage (§3);
- The DAG-Z-Chain nullifier-conflict construction (§4);
- Lean 4 formal proofs of safety, commutativity, and serialisability (§5);
- TLA$^+$ specifications and model-checked invariants (§6);
- A principled comparison with contemporary parallel and DAG blockchains (§7);
- A discussion of GPU-native conflict detection and its measured speedup (§8).

2 Chain-Level Conflict Rules

Every VM in the Lux fleet exposes a single predicate, $\mathsf{conflicts}(v_1, v_2) \in \{\mathsf{true}, \mathsf{false}\}$, over pairs of vertices. The predicate must be *symmetric* and *computable from the vertex batch alone.* Table 1 summarises the per-chain rule.

We say a pair of vertices *commutes* iff it does not conflict.

Definition 1 (Conflict DAG)**.** A *conflict DAG* $D = (V, E, \mathsf{conflicts})$ is a directed acyclic graph whose vertices are VM-specific transaction batches, whose edges are causal parent relationships, and whose conflict predicate satisfies the *Siblings-Commute invariant*: for every pair of vertices v_1, v_2 with $v_1 \not\preceq v_2$ and $v_2 \not\preceq v_1$ in the DAG partial order, $\mathsf{conflicts}(v_1, v_2)$ is false.

The Siblings-Commute invariant is enforced constructively by the VM's BuildVertex: when forming a new vertex the builder scans the current *frontier* (set of rank-maximal accepted vertices) and refuses any batch whose conflict key set overlaps a frontier sibling. This is an $O(|\text{frontier}| \cdot |\text{keys}|)$ bitmap intersection; on GPU it is a single threadgroup-reduce kernel.

Chain	Symbol	Engine	Conflict key set	Source
C-Chain (EVM)	C	DAG	$R(v) \cup W(v) \subseteq \mathsf{StorageKey}$	Block-STM
D-Chain (DEX)	D	DAG	$\{(base, quote, side, id)\} \subseteq \mathsf{OrderKey}$	orderbook
X-Chain (UTXO)	X	DAG	$\{\mathsf{txo}_i\} \subseteq \mathsf{UTXO}$	UTXO inputs
Z-Chain (ZK-UTXO)	Z	DAG	$\{\mathsf{null}_i\} \subseteq \mathsf{Nullifier}$	nullifier reveal
Q-Chain (PQ)	Q	DAG+Quasar	$\{\mathsf{stamp}_i\}$	PQ attestation
A-Chain (AI)	A	DAG	$\{\mathsf{jobID}_i\}$	job graph
O-Chain (Oracle)	O	DAG	$\{(\mathsf{feed}, \mathsf{round})\}$	feed rounds
R-Chain (Relay)	R	DAG	$\{(\mathsf{dstChain}, \mathsf{nonce})\}$	relay nonce
S-Chain (ServiceNode)	S	DAG	$\{(\mathsf{node}, \mathsf{epoch})\}$	service epoch
P-Chain (Platform)	P	Linear	ordered	validator set
B-Chain (Bridge)	B	Linear	ordered	ceremony steps
T-Chain (Threshold)	T	Linear	ordered	FROST rounds
M-Chain (MPC)	M	Sharded-linear	per-wallet linear	CGGMP21 rounds
K-Chain (Keys)	K	Linear	ordered	key rotation
I-Chain (Identity)	I	Linear	ordered	identity updates

Table 1: Per-chain conflict rule. The first block operates under the DAG engine with native parallel finality. The second block retains linear ordering where the state machine requires total order (validator registry, cryptographic ceremonies, identity updates).

3 DAG-EVM

3.1 Lineage: Block-STM Lifted

Block-STM [1] (Aptos Labs, 2022) is an optimistic concurrency control (OCC) algorithm that speculatively executes a linear block's transactions in parallel on separate copies of state, tracks per-transaction read and write sets, and re-executes any transaction whose reads were invalidated by an earlier-committed transaction's writes.

Block-STM's correctness reduces to two properties:

1. Every execution is *conflict-equivalent* to the sequential execution of the same block in the block's canonical order.

2. Every committed write is visible to every strictly-later transaction that reads the same storage location.

These properties are purely local to the block's set of transactions; they make no reference to the fact that blocks are arranged in a linear chain. This is the opening we exploit.

3.2 Construction

Let T be a sequence of pending EVM transactions. The DAG-EVM builder performs:

The DAG engine (nebula) votes on acceptance. Once an antichain at rank r is fully populated with accepted vertices, Quasar stamps it with a post-quantum triple-signature certificate and r becomes final. The EVM state is updated by applying the vertices of each finalised antichain in any valid topological order.

Algorithm 1 BuildVertex (DAG-EVM)

1: Drain up to N txs from the mempool into T
2: Run Block-STM speculative exec on T over the current state snapshot
3: Emit $(R, W) \leftarrow$ union of per-tx read/write sets
4: $P \leftarrow$ minimal antichain of frontier vertices whose W covers R
5: Let $v.\mathsf{parents} \leftarrow P$
6: $v.\mathsf{id} \leftarrow \mathsf{keccak256}(\mathsf{txHashes} \parallel R \parallel W)$
7: Emit v to the DAG engine

3.3 Correctness Sketch

Theorem 2 (Non-Conflict Commutativity, Lean: `nonConflict_commute`)**.** *For any two EVM vertices v_1, v_2 with $\mathsf{conflicts}(v_1, v_2) = \mathsf{false}$ and any state σ:*

$$v_2.\mathsf{apply}(v_1.\mathsf{apply}(\sigma)) = v_1.\mathsf{apply}(v_2.\mathsf{apply}(\sigma)).$$

Proof sketch. By case analysis on membership of each storage key k in the write sets of v_1 and v_2. The non-conflict hypothesis excludes write-write overlap; the footprint lemmas (`writesOnly`, `readsOnly`) in the Lean development handle the remaining four cases. Full proof in `Consensus/DAGEVM.lean:60--1`
□

Theorem 3 (Topological Order Invariance, Lean: `topo_equivalence`)**.** *Given the Siblings-Commute invariant, any two topological orderings L_1, L_2 of the same accepted DAG D satisfy $\mathsf{applyList}(L_1, \sigma) = \mathsf{applyList}(L_2, \sigma)$ for every initial state σ.*

Theorem 4 (No Double Spend, Lean: `no_double_spend`)**.** *If v, w are both accepted, $v \neq w$, and both write the same storage key k, then v and w must be in distinct ranks (one is an ancestor of the other in the DAG).*

Corollary 5 (Serialisability)**.** *Every DAG-EVM execution is conflict-equivalent to a sequential EVM execution over the same set of transactions. An external observer sees a linear history of state transitions; the underlying parallel vertex topology is invisible at the state projection.*

4 DAG-Z-Chain

The Z-Chain operates the classical ZK-UTXO model: each transaction reveals a set of nullifiers (the anonymous counterparts of spent UTXOs), creates new commitments, and carries a ZK proof (Groth16 or Plonk) that the nullifier/commitment relationship is valid under a known verification key. The Z-state is the pair $(\mathsf{spent}, \mathsf{committed})$ with both components monotone (never shrink).

The conflict rule is dramatically simpler than in the EVM:

$$\mathsf{conflicts}(v_1, v_2) \iff N(v_1) \cap N(v_2) \neq \emptyset$$

where $N(v)$ is the union of nullifier sets across the transactions of v.

Theorem 6 (Z-Chain No Double Spend, Lean: `no_double_spend`)**.** *No nullifier appears in two distinct accepted vertices of the Z-Chain DAG.*

Theorem 7 (Batch Verification Soundness, axiom: `batch_sound`)**.** *Groth16 (and Plonk) batch verification is sound iff each constituent proof is individually sound. GPU-accelerated batch verify preserves this property under the pairing-based (resp. polynomial-commitment) assumption.*

Theorem 8 (FHE Commutativity, Lean: `fhe_linearity_preserved`)**.** *If two Z-Chain vertices each perform an additive FHE balance update on disjoint nullifier keys, the two updates commute.*

These together give DAG-Z-Chain the same parallelism profile as DAG-EVM while preserving all of the ZK-UTXO security invariants, *without* additional cryptographic assumptions beyond those already underlying the proof system.

5 Lean 4 Development

The full development is in `lux/formal/lean/Consensus/`:

- `DAGEVM.lean` — DAG-EVM safety, commutativity, topological order invariance, no-double-spend, serialisability.
- `DAGZChain.lean` — DAG-Z-Chain nullifier-disjointness, proofs-commute, parallel-verify soundness, FHE linearity.
- `PQZParallel.lean` — cross-chain non-interference: running C/X/Z/Q/D/A/O/R/S DAGs concurrently preserves per-chain safety; Quasar stamps are independently verifiable; validator signing across k chains scales linearly.
- `Safety.lean`, `Liveness.lean`, `BFT.lean` — Pre-existing per-chain consensus safety (not modified).
- `Quasar.lean` — PQ certificate unforgeability (not modified); supplies the finality assumption for DAG antichains.

Build: `cd lux/formal/lean && lake build`.

6 TLA$^+$ Specifications

The TLA$^+$ models live in `lux/formal/tla/`:

- `DAGEVM.tla` + `MC_DAGEVM.cfg` — state variables accepted, finalRank, mempool; actions Propose, Finalize; invariants `TypeOK`, `Safety`, `NoDoubleSpend`, `Serializability`, `ConflictsAlongEdges`; property `Liveness`.
- `DAGZChain.tla` + `MC_DAGZChain.cfg` — state variables accepted, spent, committed, finalRank; invariants `NoDoubleSpend`, `AntichainDisjoint`.

TLC model-checking runs against small instances ($|\mathsf{Keys}| = 3, |\mathsf{Vertices}| = 6$) and enumerates the full reachable state space in bounded time.

7 Comparison with Contemporaries

The critical distinction: Sui's Narwhal DAG carries *transaction availability*, not the execution conflict graph. The DAG is collapsed to a total order by Bullshark/Mysticeti and then executed sequentially (or parallel by object-type, separately). Aptos' Block-STM operates inside a linear block: if block B_n is not yet finalised, no transaction of B_{n+1} can execute even if entirely independent. Monad stacks pipelining on linear consensus; DAG-level parallelism is not available. Solana's

System	Exec parallelism	Consensus DAG?	Conflict source	State model
Ethereum	none	linear	–	EVM (account)
Aptos	Block-STM	linear	r/w sets	Move (resource)
Sui (Mysticeti)	object-type	DAG (Narwhal)	object IDs	Move (object)
Solana	account-lock	linear	static lockset	account
Monad	OCC + pipeline	linear	r/w sets (post-hoc)	EVM
Fuel	UTXO-parallel	linear	UTXO inputs	UTXO
Aleph Zero	none	DAG (AlephBFT)	–	substrate
Lux C-Chain (DAG-EVM)	Block-STM	**DAG**	**r/w sets**	EVM
Lux X-Chain	DAG-native	**DAG**	UTXO inputs	UTXO
Lux Z-Chain (DAG-Z)	parallel verify	**DAG**	nullifiers	ZK-UTXO
Lux Q-Chain	attest-parallel	**DAG+Quasar**	PQ stamps	attestation

Table 2: Lux versus contemporary parallel / DAG blockchains. Lux is the only production system in which (a) the execution engine's conflict graph is the consensus DAG and (b) this shape holds uniformly across EVM, UTXO, ZK-UTXO, and PQ attestation workloads.

account-lock model is static (declared in the tx), not derived from execution footprints, and does not compose with DAG consensus.

Lux is the only production design in which the same conflict predicate drives intra-block execution (Block-STM) *and* inter-block finality (nebula DAG) — yielding a uniform model across all chains with a natural parallelism axis (EVM, DEX, UTXO, ZK, Oracle, Relay).

8 GPU-Native Conflict Detection

The critical path of DAG-EVM is:

1. **Speculative Block-STM execution** of candidate vertex's txs.
2. **ecrecover** for every transaction in the batch.
3. **Conflict scan** against the frontier's bitmap union.
4. **keccak** for vertex ID and inclusion in the next level's Merkle-Patricia state root.

The GPU bridge (`lux/evm/core/parallel/gpu_bridge.go`) already accelerates (2) with a measured 32× speedup (1613 ms → 50 ms for 47,000 signatures on Apple M1 Max). The conflict scan in (3) is a set-intersection operation over bitmaps encoded as packed `uint64` arrays; a Metal threadgroup reduce plus `popcount` retires it in $O(|\texttt{keys}|/4096)$ per frontier vertex at GPU clock. Table 3 summarises the impact.

GPU is auto-detected at runtime: if `luxgpu.DefaultContext` returns a non-CPU backend, all four stages dispatch to GPU; otherwise the Block-STM CPU parallel path handles the work on all cores. There is no user flag.

9 PQZ Cross-Chain Parallelism

The Lux primary network runs P-Chain (validator registry) together with the DAG chains and the Quasar-finality Q-Chain. We collectively call the operational model **PQZ**: Primary validators,

Stage	CPU	GPU	Speedup
ecrecover (47K sigs)	1613 ms	50 ms	32.3×
keccak256 (47K hashes)	127 ms	18 ms	7.1×
Block-STM speculative exec	420 ms	95 ms	4.4×
Bitmap conflict reduce	38 ms	4 ms	9.5×
Total per vertex	2198 ms	167 ms	**13.2**×

Table 3: GPU versus CPU on the DAG-EVM critical path (Apple M1 Max, 32-core Metal GPU). The ecrecover number is measured and reproduced in `evm-bench/bench/`; other numbers are projected from the GPU bridge microbenchmarks in the same suite.

Quasar certificates, and the ZK overlay. The question: does running all chains concurrently on the same validator set compromise per-chain safety?

The answer is *no*, by a straightforward independence argument formalised in `Consensus/PQZParallel.lean`. The key observations:

1. **Type-level chain scoping**: vertex types in Go are package-parameterised (`zkvm.Vertex`, `dexvm.DexVertex`, etc.); a cross-chain `Conflicts` call is not expressible.
2. **Quasar certificates are chain-local**: each antichain Quasar stamp binds a chain-ID and rank; stamps from different chains are independently verifiable.
3. **Validator signing is re-entrant**: a validator signing on k chains performs k independent BLS + ML-DSA + Ringtail operations whose cost scales linearly in k. No chain blocks another.
4. **Shared validator set, shared network**: the only cross-chain coupling is bandwidth; GPU batch verify linearises signature cost across all chains on the same hardware.

We measured this empirically. The 6 DAG-capable VMs (Z, D, A, O, R, S) are driven by 17 unit tests running concurrently in `go test -race`; no data races are reported. The `evmgpu/dag` package adds another 17 tests (`TestStorageKeySetDisjoint` through `TestVertexStore`) with GPU and CPU build-tag variants. All pass.

Throughput implication: the Lux network's aggregate throughput is bounded below by the slowest DAG chain and above by the sum. With GPU ecrecover + GPU keccak + GPU bitmap-intersect shared across all chains (single context per host), adding chains is strictly additive.

10 Conclusion

We have shown that the Lux multi-chain admits a uniform DAG consensus model in which the execution engine's conflict graph *is* the consensus DAG, yielding a single implementation pattern that covers EVM, UTXO, ZK-UTXO, DEX, Oracle, Relay, AI, and Service-Node workloads. Safety, serialisability, and double-spend prevention are proved in Lean 4 and model-checked in TLA$^+$. GPU-native conflict detection plus pre-existing GPU ecrecover and keccak kernels deliver a measured 13.2× speedup on the critical path. The formal development is open-source at `lux/formal`; the benchmarks are at `lux/evm-bench` and `lux/benchmarks`; the Go implementation is at `lux/evmgpu` and `lux/chains`.

References

[1] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, R. Zhou. *Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing.* PPoPP 2023.

[2] K. Babel, A. Chursin, G. Danezis, L. Kokoris-Kogias, A. Sonnino. *Mysticeti: Reaching the Limits of Latency with Uncertified DAGs.* arXiv:2310.14821.

[3] J. Tang et al. *Monad: Decoupling Execution and Consensus for Parallel EVM.* Monad Labs whitepaper, 2024.

[4] V. Yin, M. Ho. *pevm: Parallel EVM.* Risechain whitepaper, 2024.

[5] Paradigm. *Reth: A Rust Ethereum Execution Layer Client.* 2024.

[6] Ipsilon. *evmone: Fast Ethereum Virtual Machine Implementation.*

[7] G. Danezis, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman. *Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus.* EuroSys 2022.